Flutter & Dart Environment Setup

Name: Nerew Kuek Manylouny
Date: 28 September 2026
Course/Class Title: Flutter & Dart Development

Section 1: Task 1 - Flutter & Dart Setup

1. Flutter Doctor

I opened PowerShell and ran the following command to verify my Flutter installation:

flutter doctor

However, the command could not complete because the Dart SDK executable was blocked by the Windows Device Guard policy.

The terminal displayed:

'C:\flutter\bin\cache\dart-sdk\bin\dart.exe' was blocked by your organization's Device Guard policy.
Contact your support person for more info.

This indicates that Flutter is installed, but Windows is preventing the Dart SDK from running because of a Device Guard/Code Integrity security policy.

---

2. Creating the Flutter Application

I navigated to my Desktop using:

cd $HOME\OneDrive\Desktop

I then attempted to create the required Flutter application using:

flutter create my_first_app

The command returned the following message:

'C:\flutter\bin\cache\dart-sdk\bin\dart.exe' was blocked by your organization's Device Guard policy.
Contact your support person for more info.

As a result, the `my_first_app` project could not be created on my computer.

I attempted to enter the project directory using:

cd my_first_app

However, PowerShell returned an error because the project had not been created.

I also attempted to list the available Flutter devices using:

flutter devices

The same Device Guard error appeared because Flutter could not execute the Dart SDK.

---

3. Difference Between Hot Reload and Hot Restart

Hot Reload

Hot Reload allows a developer to apply changes to a running Flutter application without completely restarting the application. It generally preserves the current application state.

A developer should use Hot Reload when making quick changes to the user interface, layout, styling, or Dart code and wants to see the changes immediately.

Hot Restart

Hot Restart completely restarts the Flutter application and clears its current state.

A developer should use Hot Restart when the application needs to start again from its initial state or when Hot Reload does not properly apply a change.

Comparison

Feature	Hot Reload	Hot Restart
Restarts the application	No	Yes
Current state	Generally preserved	Cleared
Speed	Very fast	Slower than Hot Reload
Main purpose	Quickly view code/UI changes	Start the application again from the beginning

---

Conclusion

I successfully attempted the required Flutter and Dart setup commands using PowerShell. However, the commands could not complete because the Dart SDK executable was blocked by the Windows Device Guard security policy on my computer.

The issue prevented the creation of `my_first_app` and prevented Flutter from displaying the available devices. The Hot Reload and Hot Restart concepts were also reviewed as part of the task.

Section 2: Task 2 - MySQL Database Management

1. Database and Table Creation

I logged into my local MySQL server as the root user through the terminal and created a database named `school`. I then created a `students` table with an auto-incrementing primary key, student name, unique email address, and enrollment date.

SQL Script Used

```
CREATE DATABASE school;

USE school;
```

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(150) UNIQUE,
    enrolled_on DATE
);

INSERT INTO students (name, email, enrolled_on)
VALUES
('John Doe', 'john.doe@example.com', '2026-09-01'),
('Mary Wanjiku', 'mary.wanjiku@example.com', '2026-09-05');

SELECT * FROM students;
```

2. Query Results

I used the following SQL query to retrieve all records from the `students` table:

```
SELECT * FROM students;
```

The terminal returned the two student records that I inserted.

[Insert screenshot of the MySQL terminal showing the SELECT * FROM students; results here.]

3. Security Reflection

Using the MySQL `root` account directly from an application backend is poor security practice because the root account has extensive privileges over the entire MySQL server. If the application or its database credentials were compromised, an attacker could potentially perform unauthorized operations such as modifying or deleting databases, creating users, or changing privileges.

A safer approach is to create a dedicated database user for the application and give that user only the permissions required by the application. This follows the principle of least privilege and limits the potential damage if the application credentials are compromised.

The following SQL commands can be used to create a dedicated application user:

```
CREATE USER 'school_app'@'localhost'
IDENTIFIED BY 'StrongPasswordHere!';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE
ON school.*
TO 'school_app'@'localhost';

FLUSH PRIVILEGES;

SHOW GRANTS FOR 'school_app'@'localhost';
```

The `school_app` user is restricted to the `school` database and is given only the basic permissions required to read and modify student records. The application should use this account instead of the highly privileged `root` account.

Section 3: Task 3 - Python Environment & Dependency Isolation

1. Create the Project Directory

I created a project directory named `python_setup_lab` on my Desktop and worked inside the directory:

```
cd $HOME\OneDrive\Desktop
mkdir python_setup_lab
cd python_setup_lab
```

The project directory is located at:

C:\Users\snell\OneDrive\Desktop\python_setup_lab

2. Create and Activate the Virtual Environment

I created an isolated virtual environment named `venv` using:

```
python -m venv venv
```

I activated the virtual environment using:

.\venv\Scripts\Activate.ps1

The `(venv)` shown at the beginning of the PowerShell prompt confirmed that the virtual environment was active:

(venv) PS C:\Users\snell\OneDrive\Desktop\python_setup_lab>

3. Install and Verify the Requests Library

I installed the `requests` library using:

python -m pip install requests

I verified the installation using:

python -m pip show requests

The actual output confirmed:

Name: requests
Version: 2.34.2
Summary: Python HTTP for Humans.
Location: C:\Users\snell\OneDrive\Desktop\python_setup_lab\venv\Lib\site-packages
Requires: certifi, charset_normalizer, idna, urllib3

The location confirms that `requests` was installed inside the isolated `venv` environment.

I also verified the Python version:

python --version

The output was:

Python 3.13.15

4. Export the Environment Configuration

I exported the installed packages and their versions to `requirements.txt` using:

python -m pip freeze > requirements.txt

I then checked the contents of the file using:

Get-Content requirements.txt

The resulting `requirements.txt` contained:

certifi==2026.7.22
charset-normalizer==3.5.1
idna==3.20
requests==2.34.2
urllib3==2.8.0

5. Final Project Structure

I verified the project contents using:

dir

The terminal showed:

Directory: C:\Users\snell\OneDrive\Desktop\python_setup_lab

Mode	LastWriteTime	Length	Name
d-----	9/28/2026 11:29 AM		venv
-a----	9/28/2026 11:33 AM	188	requirements.txt

Therefore, the final project structure is:

python_setup_lab/
├── venv/
└── requirements.txt

Conclusion

The Python project environment was successfully created and isolated using a virtual environment named `venv`. The `requests` library was successfully installed and verified, and the environment configuration was exported to `requirements.txt`. This allows the project's dependencies to be recorded and reproduced in another Python environment.

Section 4: Task 4 - VS Code Workspace Configuration

1. Required VS Code Extensions

I configured my VS Code workspace by installing the following required extensions:

- Flutter
- Dart
- Python
- Pylance
- MySQL

The Flutter and Dart extensions were installed to provide Flutter/Dart development support, while Python and Pylance provide Python development and language-analysis features. The MySQL extension provides database development support within VS Code.

2. Python Virtual Environment Configuration

I configured VS Code to use the Python interpreter from my project's isolated virtual environment.

The selected interpreter is located at:

C:\Users\snell\OneDrive\Desktop\python_setup_lab\venv\Scripts\python.exe

The Python environment selected in VS Code is:

Python 3.13.15 ('venv')

The integrated VS Code terminal also shows the active virtual environment:

(venv) PS C:\Users\snell\OneDrive\Desktop\python_setup_lab>

3. VS Code Configuration Screenshot

The following single full-screen screenshot shows:

1. The Extensions panel with the required Flutter, Dart, Python, Pylance, and MySQL extensions installed.
2. The integrated terminal with the `(venv)` environment active.
3. The VS Code status bar showing the selected Python `venv` interpreter.